Ejercicio 04 · Clasificador visual con CNN y Model Card

INF-8239 Ciencia de Datos II · Unidad 02 · U02.E04

Edwin Ramón José Nolasco · Universidad Autónoma de Santo Domingo

Repositorio: https://github.com/rhaynel19/CV_Ejercicio-4

Síntesis ejecutiva

Se compararon un baseline denso y una CNN pequeña en Fashion-MNIST, usando la misma partición. En este test el baseline obtuvo F1 macro 0.8605 y accuracy 0.8618; la CNN obtuvo 0.7891 y 0.7918. El baseline aventajó a la CNN por 0.0714 en F1 macro y 0.0700 en accuracy.

Decisión: conservar el baseline para este benchmark y esta corrida. Aunque la CNN tiene 61.7% menos parámetros y ocupa 57.7% menos espacio, tardó 14.5 veces más en entrenar y su inferencia fue 7.2 veces más lenta. La elección se limita a este split, semilla y CPU; no es una conclusión universal.

Comparación global en el test reservado

Modelo	Accuracy	F1 macro	Parámetros	Tamaño	Train (s)	ms/imagen
Baseline denso	0.8618	0.8605	50,890	619.1 KiB	12.9	0.0254
CNN	0.7918	0.7891	19,466	262.2 KiB	187.6	0.1842

Rúbrica cubierta: auditoría y split (data_audit.json); baseline/CNN (modelos Keras); métricas, curvas y errores (cv_metrics.json y figuras); costo y decisión (tabla anterior); Model Card, pruebas, README y Git (archivos incluidos en el repositorio).

Datos, auditoría y protocolo

Fashion-MNIST contiene imágenes de prendas en escala de grises de 28 × 28 píxeles y diez clases. La fuente usada es keras.datasets.fashion_mnist; referencia del conjunto: repositorio Fashion-MNIST de Zalando Research. Cada clase está balanceada en los tres splits.

La auditoría registra semilla 42, partición estratificada y test oficial reservado. Runtime: Python 3.12.14, TensorFlow 2.21.0, Windows-11-10.0.26200-SP0, dispositivo /physical_device:CPU:0.

Particiones

Split	Imágenes	Forma	Clases	Muestras/clase	Rango
train	54,000	28 × 28	10	5,400–5,400	0–255
validation	6,000	28 × 28	10	600–600	0–255
test	10,000	28 × 28	10	1,000–1,000	0–255

El conjunto de entrenamiento de 60 000 imágenes se dividió con train_test_split estratificado: 54 000 para ajustar pesos y 6 000 para validación (10 %). Las 10 000 imágenes oficiales de test se reservaron para la evaluación final. Early stopping observó val_loss y restauró los mejores pesos; no se usó el test para seleccionar modelos.

Fingerprints SHA-256 (prefijo de 16 caracteres)

Split	Imágenes	Etiquetas
train	205525e6b5996eec	36d161bf58572d0a
validation	4ab47ec456261034	3df522c29cae234d
test	c867c93ff9536059	3d0e6c6ea990b53b

Preprocesamiento: uint8 → float32, división por 255 y canal explícito; forma del modelo (28, 28, 1), rango [0, 1]. Baseline: Flatten → Dense(64, ReLU) → Softmax(10). CNN: Conv2D(32) → MaxPooling → Conv2D(64) → MaxPooling → GlobalAveragePooling → Dropout(0.25) → Softmax(10). Ambos usaron Adam, sparse categorical cross-entropy, lote 128 y hasta 8 épocas.

Métricas por clase

Cada clase tiene soporte 1 000 en test. Se muestran precision, recall y F1 para ambos modelos; el F1 macro asigna el mismo peso a cada clase.

Clase	P base	R base	F1 base	P CNN	R CNN	F1 CNN
T-shirt/top	0.825	0.800	0.812	0.691	0.777	0.732
Trouser	0.987	0.961	0.974	0.985	0.937	0.961
Pullover	0.715	0.822	0.765	0.649	0.715	0.681
Dress	0.768	0.931	0.842	0.724	0.858	0.785
Coat	0.817	0.696	0.752	0.723	0.537	0.616
Sandal	0.961	0.948	0.955	0.948	0.891	0.919
Shirt	0.718	0.609	0.659	0.476	0.424	0.449
Sneaker	0.932	0.944	0.938	0.879	0.940	0.908
Bag	0.965	0.954	0.959	0.933	0.920	0.926
Ankle boot	0.947	0.953	0.950	0.910	0.919	0.914

Errores observados en la CNN

La CNN clasificó incorrectamente 2,082 de 10 000 imágenes. Su menor recall fue 0.424 en Shirt: 576 falsos negativos de 1,000 casos reales. El baseline alcanzó recall 0.609 en esa clase.

Clase real	Predicción	Casos
Shirt	T-shirt/top	230
Coat	Shirt	177
Shirt	Pullover	160
Coat	Pullover	159
Pullover	Shirt	141

La confusión más común fue Shirt → T-shirt/top (230 casos). La similitud visual entre prendas superiores es una explicación plausible que debe contrastarse inspeccionando los ejemplos; la matriz por sí sola no demuestra la causa.

Costo, decisión y uso responsable

En este equipo la CNN redujo parámetros en 61.7% y tamaño en 57.7%, pero elevó el tiempo de entrenamiento a 14.5× y latencia a 7.2× respecto al baseline, con menor F1 macro. Para el alcance de este benchmark, el baseline ofrece la mejor relación desempeño/costo. Antes de preferir la CNN, conviene revisar arquitectura y entrenamiento, mantener validación separada y confirmar la mejora en más de una semilla.

Alcance y limitaciones

Fashion-MNIST es un benchmark didáctico con imágenes pequeñas, centradas y monocromas. No valida fotografías reales, imágenes médicas, OCR, vigilancia ni decisiones sobre personas. Un único split/semilla no cuantifica incertidumbre y los tiempos dependen del hardware/backend. El modelo no debe desplegarse fuera del uso académico declarado.

Reproducibilidad y artefactos

Entorno: uv, Python 3.12.14 y TensorFlow 2.21.0 sobre CPU Windows 11. Las instrucciones están en README.md; requirements.txt y requirements-colab.txt declaran dependencias. La suite ejecutada informó 6 pruebas aprobadas. El notebook, código reusable, auditoría, métricas, modelos locales y figuras forman parte del proyecto.

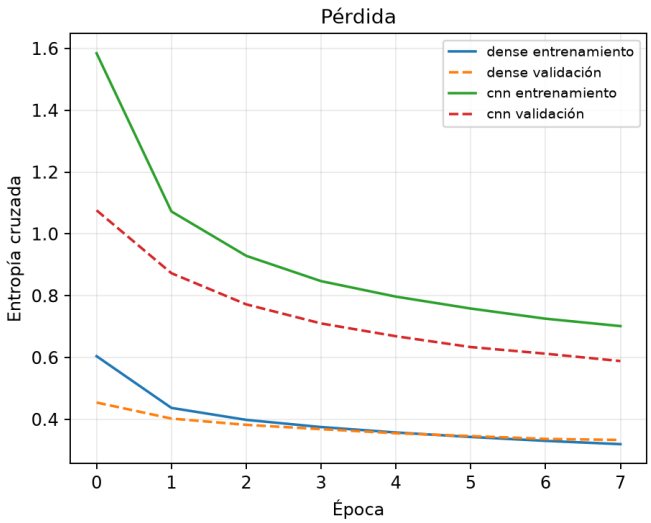
Declaración de uso de IA

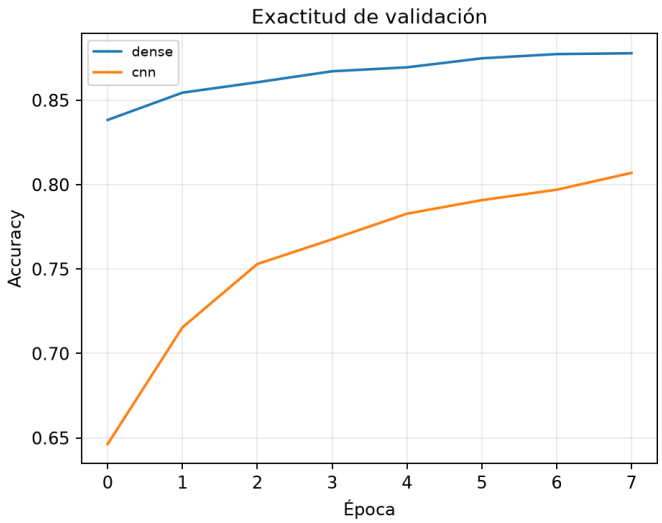
Herramienta: GitHub Copilot. Prompts relevantes: “crear el trabajo desde cero en esta carpeta” y “hazme el reporte o informe completo según la práctica”, junto con el mandato y la rúbrica de U02.E04. La asistencia apoyó el scaffold, las pruebas, el análisis de métricas y la redacción/maquetación del informe. Se verificaron instalación/runtime, pruebas, particiones, métricas, notebook JSON y artefactos. Correcciones realizadas: contrato de auditoría consumido por el PDF, filas agregadas excluidas al buscar el menor recall, anchos de tablas ReportLab corregidos, matriz de confusión persistida y política Git para binarios. El estudiante debe revisar esta declaración, confirmar que refleja su uso real y poder explicar cada decisión y resultado.

Trazabilidad con la rúbrica

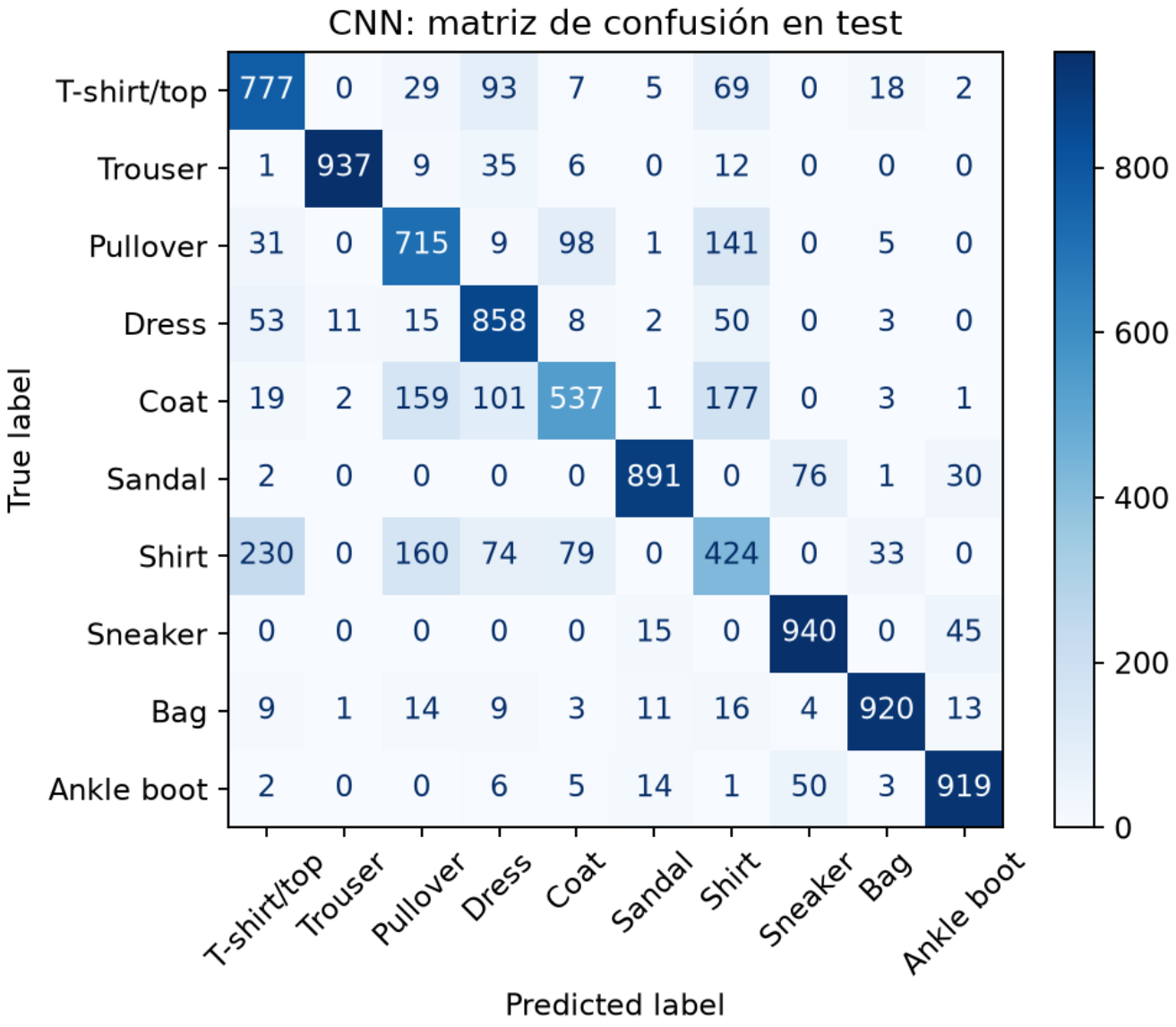
Criterio	Evidencia
Auditoría y partición reproducible	data_audit.json: split, clases, rangos, hashes y runtime.
Baseline y CNN funcional	train_cv.py, modelos Keras y test_model_contract.py.
Métricas, curvas y errores	cv_metrics.json, tabla por clase y tres figuras siguientes.
Costo y decisión técnica	Parámetros, bytes, tiempos y comparación de F1/accuracy.
Model Card, pruebas, README y Git	Archivos de entrega y repositorio enlazado arriba.

Curvas de entrenamiento y validación





Matriz de confusión de la CNN



Muestra de errores de la CNN

